

Reviewer Pack — Real Rank of Clause-Indicator Polynomial Bounds ACC⁰ Circui...

Entry f849016d205f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 20:52:18 UTC

Real Rank of Clause-Indicator Polynomial Bounds ACC⁰ Circuit Size

Entry ID: f849016d205f **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 20:52:18 UTC

1. Conjecture

Field A (mathematical branch): Real Algebraic Geometry **Field B** (complexity object): ACC⁰ Circuit Size

Statement:

For any CNF formula with n variables, the real rank of its clause-indicator polynomial over $\mathbb{R}$ is $\Theta(\log n)$ if and only if the formula can be computed by an ACC⁰ circuit of size $\Theta(n)$.

Rationale (proposer's reasoning):

Real algebraic geometry provides invariants like real rank that capture the structure of polynomial representations, which may mirror the layered complexity of ACC⁰ circuits. The real rank of the clause-indicator polynomial could reflect the minimal number of real-valued polynomial components needed to compute the CNF, directly tied to circuit size.

Taxonomy category: ACC_LB_via_WILLIAMS (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 86917e90072b0a30

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	0.95	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def gaussian_elimination(A, b):
    n = len(b)
    for i in range(n):
        max_row = i + max(range(i, n), key=lambda j: abs(A[j][i]))
        A[i], A[max_row] = A[max_row], A[i]
        b[i], b[max_row] = b[max_row], b[i]
        for j in range(i + 1, n):
            factor = A[j][i] / A[i][i]
            for k in range(i, n):
                A[j][k] -= factor * A[i][k]
            b[j] -= factor * b[i]
    x = [0] * n
    for i in range(n - 1, -1, -1):
        x[i] = (b[i] - sum(A[i][j] * x[j] for j in range(i + 1, n))) / A[i][i]
    return x

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    variables = list(range(n))
    clauses = [random.sample(variables, 3) for _ in range(random.randint(10, 20))]

    # Construct the clause-indicator polynomial
    A = [[0] * (n + 1) for _ in range(len(clauses))]
    b = [0] * len(clauses)
    for i, clause in enumerate(clauses):
        for var in clause:
            A[i][var] += 1
        b[i] = 1

    # Compute the real rank via Gaussian elimination
    rank = sum(1 for row in gaussian_elimination(A, b) if any(row))

    # Estimate ACC^0 circuit size (simplified lower bound)
    expected_rank = math.log2(n)
```

```

return {
    "metric_name": "real_rank",
    "metric_value": rank,
    "instances_tested": 1,
    "conjecture_holds": abs(rank - expected_rank) <= 0.5 * expected_rank,
    "counterexample": "" if abs(rank - expected_rank) <= 0.5 * expected_rank else f"n={n}, rea
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_dev = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_dev} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_dev} support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        counterexample = next((r["counterexample"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1932a445.py", line 68, in <module>

result = run_trial(seed)

^^^^^^^^^^^^^^^^

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1932a445.py", line 49, in run_trial

rank = sum(1 for row in gaussian_elimination(A, b) if any(row))

^^

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1932a445.py", line 25, in gaussian_elimination

factor = A[j][i] / A[i][i]

~~~~~^~~~~~

ZeroDivisionError: float division by zero

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

Test crashed with division by zero error, preventing evaluation of the conjecture’s validity  
| next: Debug gaussian\_elimination implementation to handle singular matrices

## 11. Audit log (LLM calls)

**Total LLM calls:** 9

| # | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|---|-----------------|---------------|------------------|-----------|-----|--------------|
| 1 | propose         | ollama_remote | qwen3:8b         | 0         | 0   | 109670       |
| 2 | preregistration | ollama_remote | qwen3:8b         | 0         | 0   | 32439        |
| 3 | novelty         | ollama_remote | qwen3:8b         | 0         | 0   | 30768        |
| 4 | novelty         | ollama_remote | qwen3:8b         | 0         | 0   | 22773        |
| 5 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 20020        |
| 6 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 11318        |
| 7 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 10168        |
| 8 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 9689         |
| 9 | judge           | ollama_remote | qwen3:8b         | 0         | 0   | 104224       |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 351067 ms total latency. Provider mix: {'ollama\_remote': 9}

*(full prompt+response transcripts available in research/audit/f849016d205f.jsonl)*

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/f849016d205f.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** research/pvsnp\_sandbox/test\_\*.py (per-cycle)

**Replay tarball:** research/replay/f849016d205f.tar.gz (if generated)